DW TRAVEL ADVISER

CLI — Project Report

Plan Smarter. Travel Better.

Author: Dhruv Prasad Warrier.

Language: Python 3.x

Date: March 28, 2026

1. Introduction

DW Travel Adviser is a command-line interface (CLI) application developed in Python that helps users plan international trips from India. The application allows users to compare travel destinations, view real-time simulated flight prices, calculate total trip costs, and receive smart destination recommendations based on their budget.

The project was developed as a personal utility tool and demonstrates the use of core Python programming concepts including modular design, file I/O, data manipulation with pandas, password hashing, and structured CLI interaction.

2. Objectives

- Provide a terminal-based travel planning tool accessible to any Python user
- Implement a secure user authentication system with login and signup
- Separate flight rate data from the CSV for easy maintenance and future API integration
- Offer multiple fare class options (Economy, Standard, Flexible, Business) with simulated live pricing
- Calculate detailed, accurate cost breakdowns distinguishing between per-person and per-room costs
- Recommend the best destinations based on user budget, ranked by safety and cost

3. Project Structure

The project is divided into four Python modules and one data file, each with a clear responsibility:

File	Type	Responsibility
main.py	Entry Point	Displays banner, handles auth menu loop, launches planner after login
auth.py	Module	User signup and login with SHA-256 password hashing, 3-attempt limit
traveladviser.py	Module	Trip planner and smart recommender — reads CSV, calculates costs, displays results
flight_rates.py	Module	Stores base flight prices, simulates hourly rate fluctuation, presents fare class options
Travel_list.csv	Data File	Hotel, misc, visa costs, safety rating, currency, best season for 20 destinations
users.json	Data Store	Auto generated on first signup; stores hashed credentials and user profiles

4. Module Descriptions

| 4.1 main.py — Entry Point

The entry point of the application. It prints a welcome banner and presents a loop with three options: Login, Sign Up, and Exit. On successful login, it passes control to the travel planner. This separation ensures authentication is always enforced before accessing planner features.

| 4.2 auth.py — Authentication

Handles all user account operations. Passwords are never stored in plain text — they are hashed using the SHA-256 algorithm from Python's built-in hashlib module before being written to users.json. Key features include:

- Email, first name, last name, username, and password collected at signup
- Duplicate username check before account creation
- Login enforces a maximum of 3 attempts before denying access
- getpass is used for password input so characters are not echoed to the terminal

| 4.3 flight_rates.py — Flight Rate Engine

This module stores base flight prices (in INR from India) for all 20 destinations separately from the CSV. This design decision makes it easy to swap in a real flight API in the future without touching any other file. Features include:

- Hourly price fluctuation of up to +/- 8% simulated using time-seeded arithmetic
- Four fare tiers: Economy (best deal), Standard (most popular), Flexible (changeable), and Business
- Prices rounded to the nearest INR 500 for realistic presentation
- Recommended tier labels shown to guide user choice

| 4.4 traveladviser.py — Planner & Recommender

The core module with two main features:

Plan a Trip

Displays all 20 destinations with safety ratings and a short description. After the user selects a country, the module calls flight_rates.py to show live fare options. The user then inputs the number of persons, rooms, and nights. Costs are calculated as follows:

Cost Component	Formula	Rationale
Flight	Flight price x Persons	Each person needs a ticket
Hotel	Hotel/night x Rooms x Nights	Rooms are shared; billed per room
Misc Expenses	Misc/day x Persons x Nights	Each person spends daily
Visa	Visa cost x Persons	Each person needs an individual visa
Grand Total	**Sum of all above**	Displayed with per-person breakdown

Smart Recommender

The user enters a total budget, number of persons, and number of nights. The engine calculates the economy-class total cost for all 20 destinations and filters those within budget. Results are ranked first by safety rating (highest first), then by total cost (lowest first). The top 5 are displayed with remaining budget shown for each.

5. Data — Travel_list.csv

Flight prices have been intentionally removed from the CSV and moved into flight_rates.py. This separation provides a cleaner data model — the CSV now contains only destination attributes that change rarely, while flight prices (which fluctuate frequently) are managed in code. The CSV contains the following columns:

Column	Description
Country	Destination name
Avg Hotel per Night (INR)	Average cost of one hotel room per night in INR
Misc Expenses per Day (INR)	Average daily spending per person (food, transport, activities)
Visa Cost (INR)	Visa fee per person in INR
Safety Rating (1-10)	Safety score; >= 8 is considered safe, < 7 triggers a caution advisory
Currency	Local currency code (e.g. JPY, EUR, USD)
Currency Rate (1 unit to INR)	Exchange rate for reference
Best Time to Visit	Recommended travel months
Known For	Short highlight of what the destination is famous for

The application currently includes 20 destinations: Japan, France, USA, UAE, Thailand, Australia, Germany, Singapore, Canada, Italy, Spain, Netherlands, South Korea, Turkey, Malaysia, Indonesia, UK, Switzerland, New Zealand, and Vietnam.

6. Technologies Used

Technology	Version	Usage
Python	3.7+	Core programming language
pandas	Latest	Reading and querying the CSV data file
hashlib	Built-in	SHA-256 password hashing for secure auth
getpass	Built-in	Hidden password input in terminal
json	Built-in	Reading and writing users.json
datetime	Built-in	Hour-based seed for flight rate simulation
os	Built-in	File path and existence checks

7. How to Run

Ensure all six files are in the same folder. Install the only external dependency if not already present:

```
pip install pandas
```

Then run the application:

```
python main.py
```

The __pycache__ folder generated by Python is not part of the project and should not be shared. The users.json file is auto created on the first signup.

8. Sample User Flow

- User launches the app with python main.py
- User selects Sign Up and creates an account with name, email, username, and password
- User logs in — password is verified against the SHA-256 hash
- Main menu appears: Plan a Trip or Smart Recommend
- Plan a Trip: user picks a country, selects Economy fare, enters 2 persons / 1 room / 5 nights

- App displays full cost breakdown: flight, hotel, misc, visa, grand total, per-person cost, safety advisory
- Smart Recommend: user enters budget INR 3,00,000, 2 persons, 7 nights
- App lists all destinations within budget, ranked by safety — shows top 5 with remaining budget

9. Limitations & Future Scope

| Current Limitations

- Flight prices are simulated — not pulled from a real API
- No GUI; purely terminal-based
- users.json is stored locally — no multi-device sync
- No trip history or saved itineraries

| Future Improvements

- Integrate a real flight search API (e.g. Amadeus, Skyscanner) for live prices
- Add trip history so users can view and compare past calculations
- Export trip summary as a PDF or text file
- Add multi-city trip support
- Build a simple GUI using tkinter or a web front-end

10. Conclusion

DW Travel Adviser is a well-structured, modular Python CLI application that solves a real-world problem — helping travellers plan and estimate the cost of international trips from India. The project demonstrates good software design principles including separation of concerns, secure credential management, and accurate financial calculations.

The separation of flight data from the CSV, the introduction of fare class tiers, and the smart budget-based recommender represents significant improvements over the original version. The application is ready to use and provides a solid foundation for future enhancements such as live API integration and trip history management.

— End of Report —